

SYSTEM DOCUMENTATION

File Net

A Firebase-backed Flutter File Sharing & Document Management Platform

Prepared: July 7, 2026

Scope: Architecture, authentication, data model, feature walkthroughs, admin operations, and known issues.

Source: `file_sharing_master` Flutter codebase

Table of Contents

1. Overview & Purpose

2. Technology Stack

3. System Architecture

4. User Roles & Permissions

5. Authentication & Session Management

6. Data Model

7. Application Routes

8. Feature Walkthrough

9. Notifications (SMS)

10. Admin Access

11. Known Issues & Recommendations

12. Appendix — Firebase Resources

1. Overview & Purpose

File Net is a cross-platform Flutter application (Android, iOS, Web, macOS, Windows, Linux) for sharing PDF documents between organizational users. Its core concept is the **folder** ("File"): a titled, described container that an owner creates, populates with uploaded PDF documents, and shares with specific colleagues. Access to each folder is restricted to its creator, the users it has been explicitly shared with, and platform administrators.

The system supports three functional roles — **Admin**, **Office**, and **Lecturer** — reflecting its origin as an internal document-distribution tool for an academic or office environment (offices push notices/documents down to lecturers, and lecturers/offices share documents with each other). Administrators additionally get a management console for overseeing all users, offices, and folders platform-wide.

There is no push-notification service; instead, the app notifies users of new uploads or newly shared folders via SMS through a third-party gateway (Arkesel).

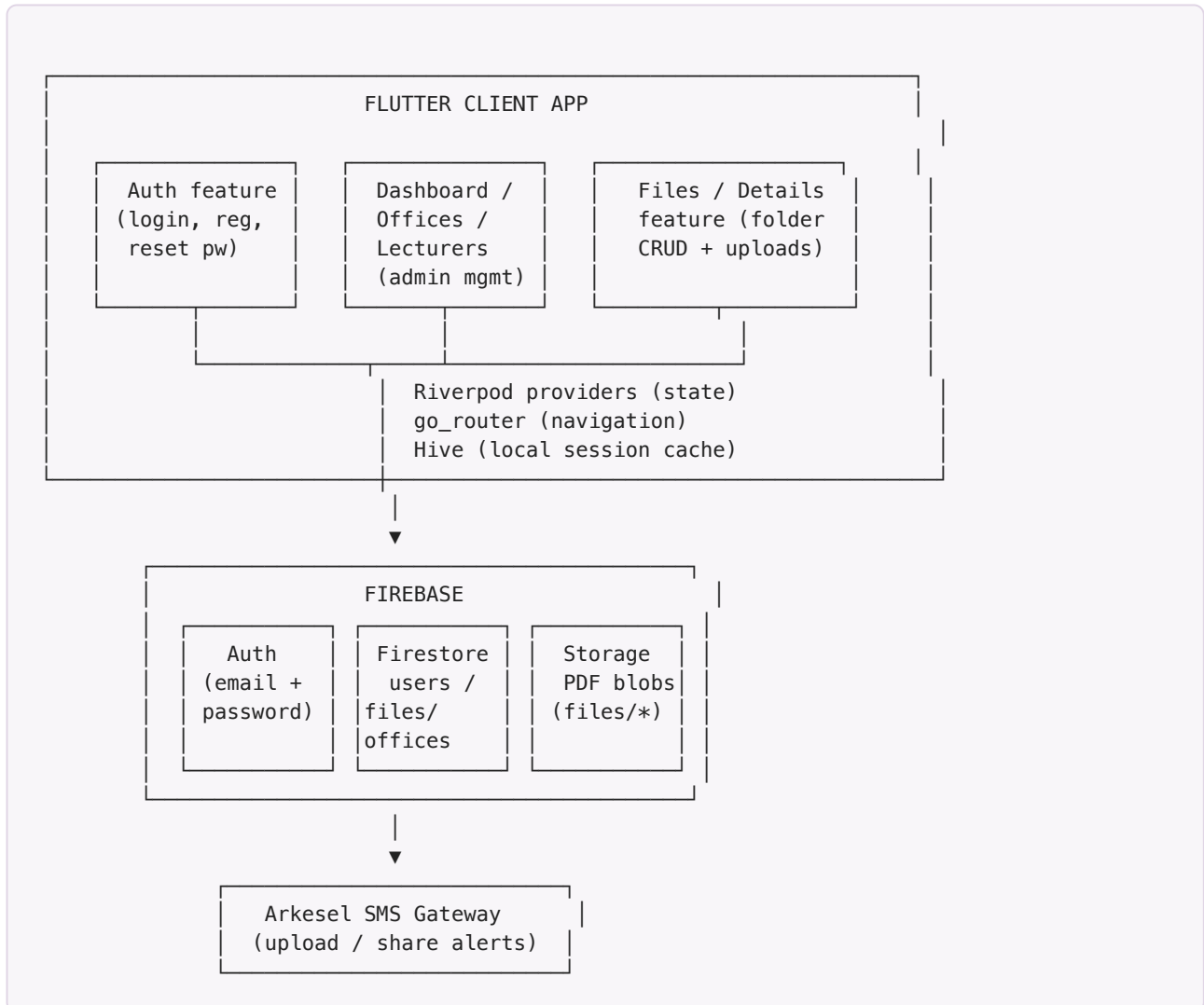
2. Technology Stack

Layer	Technology	Notes
Client framework	Flutter (Dart)	Single codebase targeting mobile, web, and desktop
State management	flutter_riverpod 2.6.1	StateNotifierProvider / StreamProvider pattern throughout
Routing	go_router 16.0.0	Declarative routes with two shell routes (auth shell, app shell)
Authentication	firebase_auth 5.6.2	Email/password sign-in; email verification required for non-admins
Database	cloud_firestore 5.6.11	Collections: <code>users</code> , <code>files</code> , <code>offices</code>
File storage	firebase_storage 12.4.9	Stores uploaded PDF binaries and (unused) profile images
Local persistence	hive_flutter 1.1.0	Persists the logged-in user's session across app restarts
File selection	file_picker 10.2.0	Restricted to <code>.pdf</code> uploads, 5 MB size cap
Notifications	Arkesel SMS API (via <code>http</code>)	Custom REST call — no Firebase Cloud Messaging is used
UI feedback	flutter_smart_dialog	Loading spinners, toasts, and confirm dialogs
Admin tables	data_table_2	Sortable/searchable tables for Offices & Lecturers management
Fonts	google_fonts (Poppins, Roboto, Open Sans)	Responsive type scale via <code>Styles(context)</code> helper

Unused dependencies: `syncfusion_flutter_pdfviewer` and `file_saver` are declared in `pubspec.yaml` but not actively used in the current UI — PDFs are opened via external browser/viewer through a direct download link rather than an in-app viewer.

3. System Architecture

File Net follows a simple client-driven architecture with no custom backend server — Firebase acts as the entire backend, and a third-party SMS API supplements it for notifications.



Firebase project: [file-net-97e98](#). There is no Cloud Functions / server-side logic layer — all business rules (role checks, folder visibility, ban enforcement, SMS triggers) execute client-side inside the Flutter app.

4. User Roles & Permissions

Role	Created by	Access
admin	Seeded directly in Firebase (no in-app admin creation flow)	Full platform access: dashboard stats, all folders (view/upload/share/delete regardless of ownership), Offices management, Lecturers/Users management (block/unblock). Bypasses the email-verification requirement at login.
office	Created by an admin via Offices management (auto-generates a login)	Same folder capabilities as a Lecturer: create/upload/share folders, act on folders they own or have been shared. No access to admin management screens.
lecturer	Self-registration via the public Register page (default role)	Create folders, upload PDFs, share folders with other users, edit/delete only folders they created, view/download files shared with them.

Authorization gap: role restrictions are enforced only by hiding menu items in the UI. The `go_router` configuration does not implement a route guard/redirect based on `user.role`, so a non-admin user who manually navigates to `/dashboard`, `/admin/offices`, or `/lecturers` is not blocked at the routing layer. Firestore security rules (not reviewed here) may or may not compensate for this at the data layer — this should be verified independently.

5. Authentication & Session Management

5.1 Registration

1. User submits name, email, phone, and password on the Register page.
2. `AuthServices.createUser` calls Firebase Auth's `createUserWithEmailAndPassword`, sends a verification email, and writes a `UserModel` document to Firestore (`users/{uid}`) with `role: 'lecturer'` and `status: 'active'`.
3. The user is immediately signed back out and redirected to Login with a "please verify your email" message.

5.2 Login

1. Email + password submitted → Firebase `signInWithEmailAndPassword`.
2. The matching `users/{uid}` Firestore document is fetched.
3. **Verification check:** login is rejected unless `emailVerified == true` — except for accounts with `role == 'admin'`, which bypass this check.
4. **Ban check:** if `status == 'banned'`, login is rejected with a message to contact the administrator.
5. On success, the user record is stored in the Riverpod `userProvider` and mirrored into Hive local storage (key `'user'`) for session persistence across restarts.
6. **Redirect:** `role == 'admin'` → `/dashboard`; every other role → `/files`.

5.3 Session Restore

On app launch, the stored Hive session is read and the corresponding Firestore user document is re-fetched, which re-validates ban status — a banned user is effectively logged out on their next app open, not just at explicit login time.

5.4 Password Reset & Logout

The Forgot Password page triggers Firebase's standard `sendPasswordResetEmail` flow. Logging out clears the Hive session, calls Firebase `signOut()`, and routes back to `/login`.

6. Data Model

All data lives in three Firestore collections. Uploaded documents are embedded inside their parent folder rather than stored as a separate collection.

6.1 UserModel (collection: users)

Field	Type	Notes
id	string	Firebase Auth UID
name	string	
email	string	
phone	string	Used as the SMS notification destination
role	string	<code>admin</code> / <code>office</code> / <code>lecturer</code> (default)
password	string	⚠ Stored in plaintext on the Firestore document — see §11
status	string	<code>active</code> (default) or <code>banned</code>
createdAt	int (epoch ms)	

6.2 FileModel — "Folder" (collection: files)

Field	Type	Notes
id	string	
title	string	
description	string	
creatorId	string	UID of the folder's owner
creator	map	Embedded snapshot of the creator's UserModel
userIds	array<string>	UIDs with access to this folder
users	array<map>	Embedded UserModel snapshots for each shared user
files	array<map>	Embedded <code>Upload</code> objects (see 6.3)
status	string	Default <code>opened</code>
createdAt	int (epoch ms)	

6.3 Upload (embedded in FileModel.files)

Field	Type	Notes
id	string	Timestamp-based ID, also the Storage object key
description	string	

fileUrl	string	Firebase Storage download URL
folderId	string	Parent FileModel id
uploadBy	string	Uploader's UID
createdAt	int (epoch ms)	

6.4 OfficeModel (collection: offices)

Field	Type	Notes
id	string	⚠ Not reconciled with the linked Auth user's UID — see §11
name	string	
officeEmail	string	Also used as the office's login email
officePhone	string	
password	string	Left blank in the creation UI; the linked Auth login instead defaults to <code>123456</code>
status	string	
createdAt	int (epoch ms)	

7. Application Routes

Path	Screen	Shell	Intended audience
<code>/login</code>	Login Page	Auth shell	Everyone (public)
<code>/register</code>	Registration Page	Auth shell	Everyone (public)
<code>/forgot-password</code>	Forgot Password Page	Auth shell	Everyone (public)
<code>/dashboard</code>	Dashboard Home (stat tiles)	App shell	Admin
<code>/admin/offices</code>	Offices Management	App shell	Admin
<code>/lecturers</code>	Lecturers / Users Management	App shell	Admin
<code>/files</code>	Files (folders) list	App shell	All authenticated roles
<code>/file/:fileId</code>	File Details (documents in a folder)	App shell	Folder creator, shared users, admin

`initialLocation` is `/login` ; the app shell wraps the sidebar/app-bar navigation (`DashBoardMainPage`) around every authenticated route.

8. Feature Walkthrough

8.1 Dashboard (Admin Home)

The landing page after an admin logs in shows three live, clickable stat tiles pulled from Firestore in real time: total **Files** (folders), total **Offices**, and total **Lecturers** (all non-admin users). Clicking a tile navigates to that resource's management page.

8.2 Files — Folder Management (all roles)

The Files page is shared across every role, but the data each user sees differs:

- **Admins** see every folder in the system.
- **Office / Lecturer** users see only folders they created or that have been explicitly shared with them (i.e. their UID appears in `usersIds`).

From this list, a user can:

- **Create a folder** — supply a title and description.
- **Search/filter** folders by title.
- **Open** a folder to view its details and documents (see §8.4).
- **Upload a PDF** into any folder they have access to (see constraints below).
- **Share** a folder with other users, selecting from a dropdown of everyone except themselves.
- **Edit or delete** a folder — restricted to the original creator only.

Upload constraints

Rule	Detail
File type	PDF only, enforced via <code>file_picker</code> 's type filter
Size limit	5,000,000 bytes (~5 MB) — the on-screen error message incorrectly states "1MB"
Storage path	<code>files/{timestampId}</code> in Firebase Storage
Side effects	Uploader is auto-added to the folder's <code>users</code> / <code>usersIds</code> if not already present; the folder creator receives an SMS if someone else uploaded

8.3 Sharing a Folder

Sharing appends the selected users to the folder's `users` / `usersIds` arrays and triggers an SMS to each newly added user, notifying them a folder has been shared with them.

8.4 File Details Page

Opening a folder (`/file/:fileId`) shows its metadata — title, description, creator name, email, phone, and creation date — followed by a step-indicator list of every uploaded document (description, uploader name, upload date) with a **Download** button. Downloading opens the Firebase Storage URL directly via the device's default browser/handler (`url_launcher`); there is no in-app PDF viewer despite the Syncfusion PDF viewer package being present in dependencies.

8.5 Offices Management (Admin only)

A searchable, sortable table (`/admin/offices`) of all Office entities with full CRUD:

- **Create** — admin enters a name, email, and phone. This simultaneously (a) creates an `OfficeModel` Firestore document and (b) provisions a new Firebase Auth login for that office with `role: 'office'` and a hardcoded default password of `123456` .
- **Edit** — updates the office's stored fields.
- **Delete** — removes the Office Firestore document (does not remove the associated Auth account/login).

8.6 Lecturers / Users Management (Admin only)

A searchable table (`/lecturers`) listing every non-admin user in the system, with a **Block / Unblock** action per row. Blocking flips a user's Firestore `status` field to `banned` , which is enforced both at next login and on session restore (a banned user is force-logged-out the next time they open the app).

9. Notifications (SMS)

File Net has no push-notification service (no Firebase Cloud Messaging integration). Instead, it notifies users of relevant activity via SMS through a third-party gateway, **Arkesel**, called directly from the client using the `http` package.

Trigger	Recipient
A user uploads a document into a folder they don't own	The folder's creator
A folder is shared with a user	Each newly added user

The Arkesel API key is hardcoded directly in application source (`lib/core/functions/sms_gpt_model.dart`) rather than stored in a secrets manager or environment configuration — see §11.

10. Admin Access

An administrator account exists for managing the platform end-to-end (dashboard, offices, lecturers, and unrestricted folder access). Current credentials for this environment:

Field	Value
Login URL	<code>/login</code> (app entry point)
Email	<code>admin@gmail.com</code>
Password	<code>123456</code>
Role	<code>admin</code>

Security note: this is a weak, easily-guessable password on a privileged account, and (per §6.1) user passwords are stored in plaintext on the Firestore `users` collection rather than relying solely on Firebase Auth's hashed credential store. Treat these credentials as sensitive — avoid distributing this document outside people who need admin access, and consider rotating the password and auditing Firestore security rules before any production/public release.

After logging in as admin, the sidebar (desktop) or app-bar menu (mobile/tablet) exposes: **Dashboard**, **Offices**, **Lecturers**, and **Files** — covering every capability described in §8.

11. Known Issues & Recommendations

#	Issue	Location	Recommendation
1	No route-level role guard — hidden menu items are the only thing stopping a non-admin from navigating directly to admin routes.	<code>lib/router/router.dart</code>	Add a <code>redirect</code> callback that checks <code>user.role</code> against each route's required role and bounces unauthorized users to <code>/files</code> .
2	User passwords stored in plaintext in the Firestore <code>users</code> document, duplicating what Firebase Auth already stores securely (hashed).	<code>UserModel.toMap()</code>	Remove the <code>password</code> field from the Firestore document entirely; rely on Firebase Auth as the sole credential store.
3	Office creation doesn't reconcile the generated <code>OfficeModel.id</code> with the UID of the Firebase Auth account created alongside it, so the office record and its login are only loosely linked.	Offices "Create" flow	Use the Auth-generated UID as the Office document's ID (or store it explicitly as a <code>userId</code> field) at creation time.
4	Editing an Office calls <code>AuthServices.updateUser</code> with an empty user ID, so the linked login/user record is not actually updated.	Offices "Edit" flow	Pass the office's actual linked user ID through to the update call.
5	Upload size-limit error message says "1MB" while the enforced limit is ~5MB.	Upload validation copy	Correct the message text to match the real 5,000,000-byte limit.
6	Profile editing and avatar upload have working backend methods but no UI ever calls them — dead/unreachable functionality.	<code>AuthServices.updateProfile</code> , <code>uploadUserImage</code>	Either wire up a profile-editing screen or remove the unused code.
7	Unused dependencies (<code>syncfusion_flutter_pdfviewer</code> , <code>file_saver</code>) and dead code (<code>UserRole</code> enum, <code>FooterPage</code>)	Various	Remove unused packages/classes, or finish wiring them in (e.g. an in-

	widget) increase bundle size and maintenance surface without benefit.		app PDF viewer instead of external download).
8	Arkesel SMS API key is hardcoded in client source, shippable inside the compiled app and extractable by anyone who decompiles it.	<code>lib/core/functions/sms_gpt_model.dart</code>	Move SMS sending behind a server-side function (e.g. a Cloud Function) so the key never ships to the client.
9	Default office password (<code>123456</code>) is identical to the current admin account's password and is never forced to be changed on first login.	Offices "Create" flow	Force a password reset/change on first login for auto-provisioned accounts.

12. Appendix — Firebase Resources

Firestore Collections

Collection	Purpose
<code>users</code>	All accounts — admin, office, and lecturer roles
<code>files</code>	Shareable folders, each with embedded uploaded-document metadata
<code>offices</code>	Office entities linked to an auto-provisioned login

Firebase Storage Paths

Path pattern	Contents	Status
<code>files/{uploadId}</code>	Uploaded PDF binaries	Active
<code>users/{id}.jpg</code>	User profile photo	Backend method exists; no UI triggers it (dead code)

Firebase Project

Project ID: `file-net-97e98`. Configured services: Authentication, Firestore, Storage. No Cloud Functions, Cloud Messaging, or Analytics packages are present in this codebase.

Brand Reference

Token	Value
App title	File Net
Primary color	<code>#50006C</code> (deep purple)
Secondary color	<code>#EFA95E</code> (amber)
Typography	Poppins (titles), Roboto (subtitles), Open Sans (body) via Google Fonts